42 ABU DHABI · CAPSTONE PROJECT · STAFF EVALUATION

Capstone Project: Ft_Transcendence

Salim Hamzaoui · shamzaou

Alisher Abdullaev · alabdull

Nasser Alzaabi · naalzaab

Nour Murat · nurmurat

FAST_PONG — a web platform for 3D Pong and Tic-Tac-Toe with online matchmaking, tournaments, player statistics, 42 login, 2FA and GDPR tools

7 Major + 6 Minor modules = 10 major-equivalents · Evaluation date: ____ / ____ / 2026

CONTENTS

01

Introduction

Presenter: **Nour Murat**

Project Overview

Ft_transcendence is a web-based multiplayer gaming platform developed as the capstone project of the 42 Abu Dhabi curriculum. Its core is a modern 3D implementation of the classic Pong, joined by a second game — Tic-Tac-Toe with online matchmaking — and a complete user experience: secure accounts, 42 Intra login, email two-factor authentication, player profiles with statistics and match history, a friends list, a tournament system and GDPR tools.

The application is a Single Page Application: Django renders the first page on the server, then JavaScript swaps views without full reloads. The backend is Django (Python) with a PostgreSQL database; Gunicorn serves the site over HTTPS, and the whole stack runs in Docker Compose. The 3D game is rendered with Three.js (WebGL) and includes a computer-controlled opponent.

Backend

Django 4.2, Django REST Framework, SimpleJWT, Gunicorn (HTTPS on port 443)

Frontend

Vanilla JavaScript SPA, Bootstrap toolkit, Three.js for the 3D Pong scene

Data & Ops

PostgreSQL 13, Docker Compose, Git/GitHub feature-branch workflow

Project Objectives

The goal was to design, develop and deploy a fully functional and secure web application centred on a Pong game. The objectives set at the outset were:

Functional gaming platform

A fully operational website with 3D Pong (two players or versus AI) and a second game, Tic-Tac-Toe, playable locally or online.

Secure authentication

Email/password registration, 42 Intra OAuth for students, optional email-based 2FA and JWT tokens.

Persistent user profiles

Display name, avatar, friends, win/loss statistics, best score and a complete match history for both games.

Tournament mode

Create a tournament of 3–8 nicknames, play a round-robin of Pong matches and determine a winner.

Application security & GDPR

Protection against SQL injection, XSS and CSRF; data export, anonymization and account deletion.

Collaborative development

An Agile, feature-branch workflow with code reviews, Docker and a maintainable codebase.

Scope of the Project

The scope covers every essential aspect of a modern web application, from user management to gameplay and deployment:

User management

Registration, login, 42 OAuth, profile editing, avatar upload, friends list.

Pong (3D)

Local two-player mode on one keyboard and a single-player mode against the AI opponent.

Tic-Tac-Toe

Second game: local mode, or online against another logged-in user found by matchmaking.

Tournaments

Creation, nickname registration, automatic match generation, tiebreakers, winner.

Profiles & statistics

Games played, win rate, best score, recent matches, JSON export of all data.

Security & privacy

Hashed passwords, HTTPS, CSRF, 2FA + JWT, GDPR anonymize / export / delete.

Deployment

Two containers (web, db) orchestrated with Docker Compose; one command to run.

Team process

Agile iterations, Git feature branches, pull requests and peer review.

02

Software Development Life Cycle

Presenter: **Alisher Abdullaev**

Chosen SDLC Model: Agile (iterative & incremental)

The project was built in short cycles, each delivering one working feature that was merged and tested before the next one started. This suited a four-person learning project with evolving requirements.

1 Iterative development

Work was broken into features — authentication, Pong core, tournaments, profiles — each built in its own short cycle.

2 Incremental delivery

A runnable, testable version existed after every merge; functionality grew with each iteration.

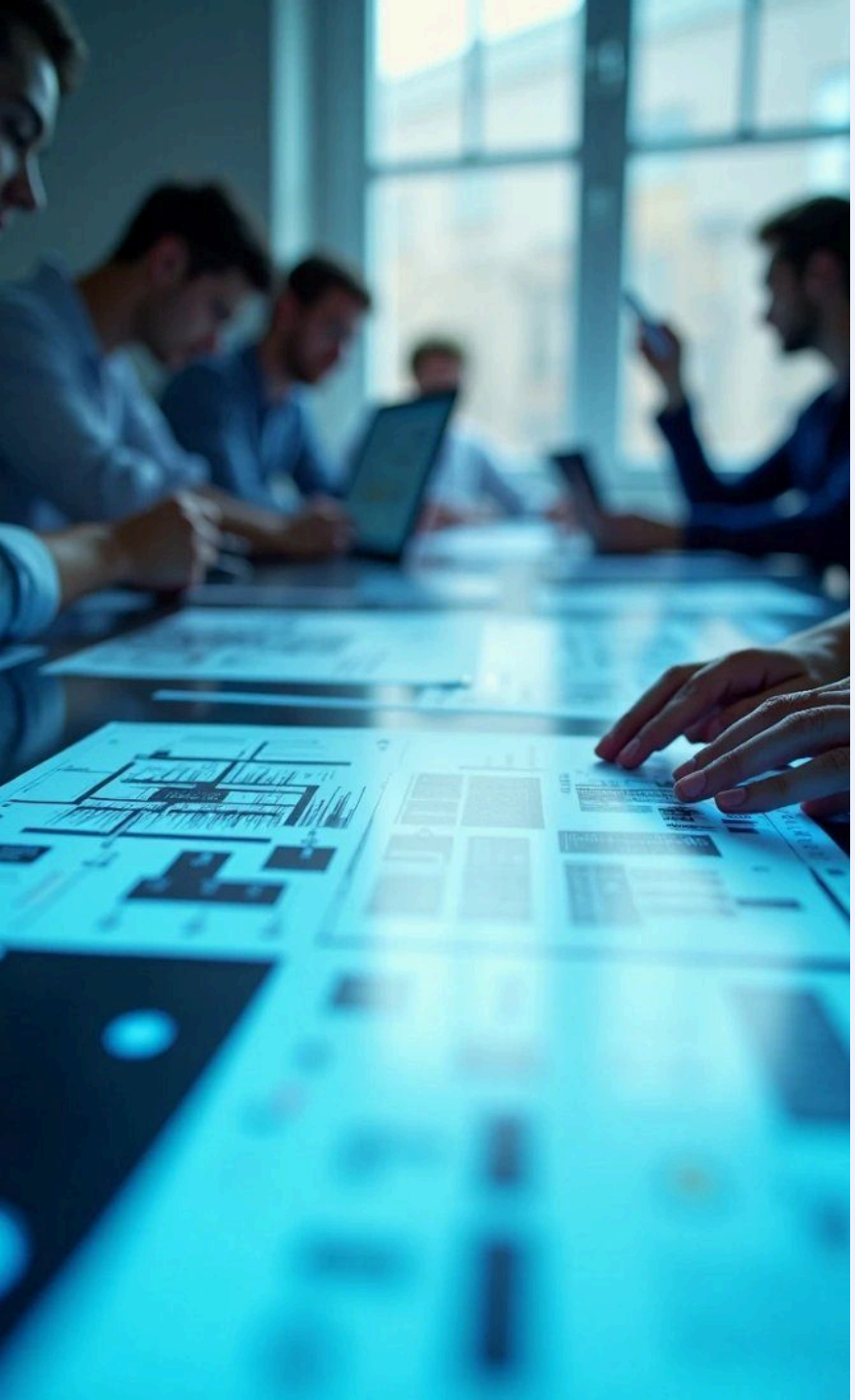
3 Flexibility

Requirements and the technical approach were refined as our understanding of the project grew.

4 Collaboration & feedback

Daily check-ins, pairing on complex features and peer review of every merge.

Evidence in the repository: 86 commits between 10 Feb and 2 Apr 2025, 15 pull requests merged from feature branches (db-connect, game-setup, tournaments, profile-page, secure-cookies, OAuth, user-settings, delete-account …), plus the 2026 pre-evaluation sweeps.



Phases and Iterations

We did not use formal time-boxed sprints, but every major feature went through the same cycle of phases:

1 Planning

Discuss the next feature (e.g. 2FA, tournament logic), clarify objectives and acceptance criteria.

2 Design

Sketch data models, API endpoints and simple wireframes before coding.

3 Implementation

Develop the feature in a dedicated Git branch to avoid conflicts with the main codebase.

4 Developer testing

Unit tests for backend logic and functional checks by the developer.

5 Integration & review

Peer code review, then merge of the feature branch into master.

6 System testing

Test the feature inside the whole application to catch regressions.

Team Collaboration and Version Control

Clear processes and modern tools kept four developers working in parallel without stepping on each other.

Version control

Git with GitHub as the central repository — every change tracked and reversible.

Branching strategy

Feature-based branches (OAuth, tournaments, profile-page …) isolated work in progress from master.

Code reviews

Every branch was reviewed through a pull request before merging — 15 PRs in total.

Communication

A WhatsApp group for quick coordination and regular in-person meetings on campus.

Project Timeline (Gantt Chart)

The Gantt chart planned and tracked the project over time: task durations, dependencies and milestones.

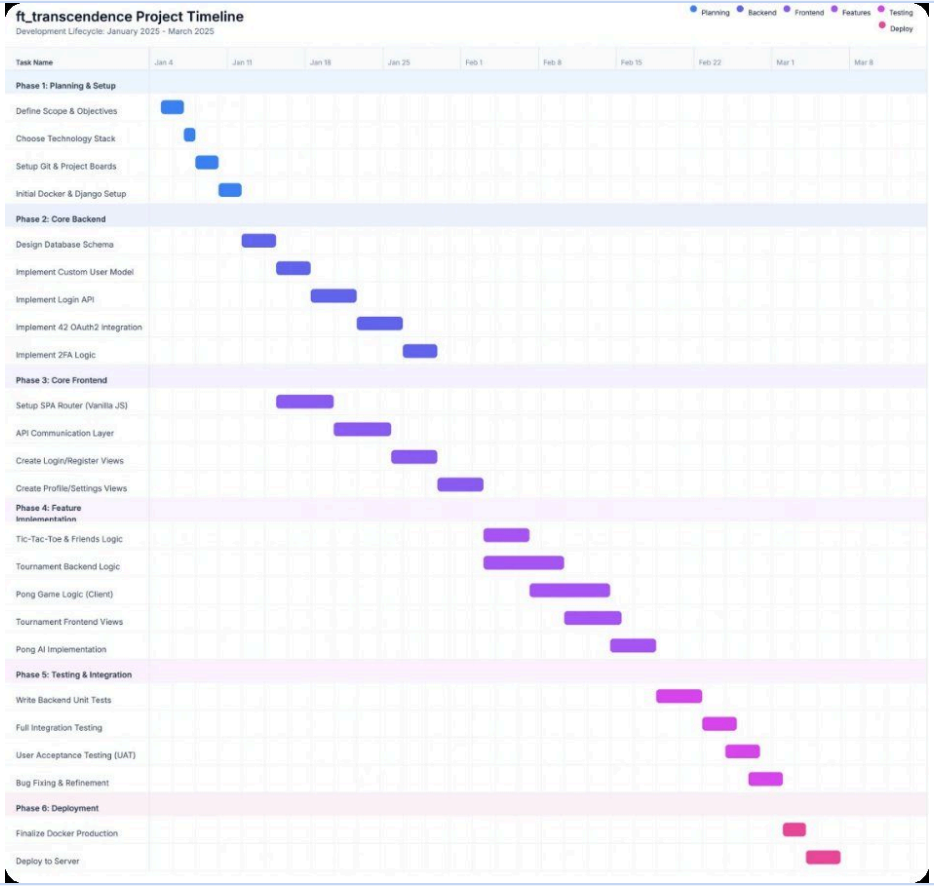


Figure 1: Project Gantt chart — planning & setup, core frontend, core backend, feature implementation, testing & integration, deployment.

03

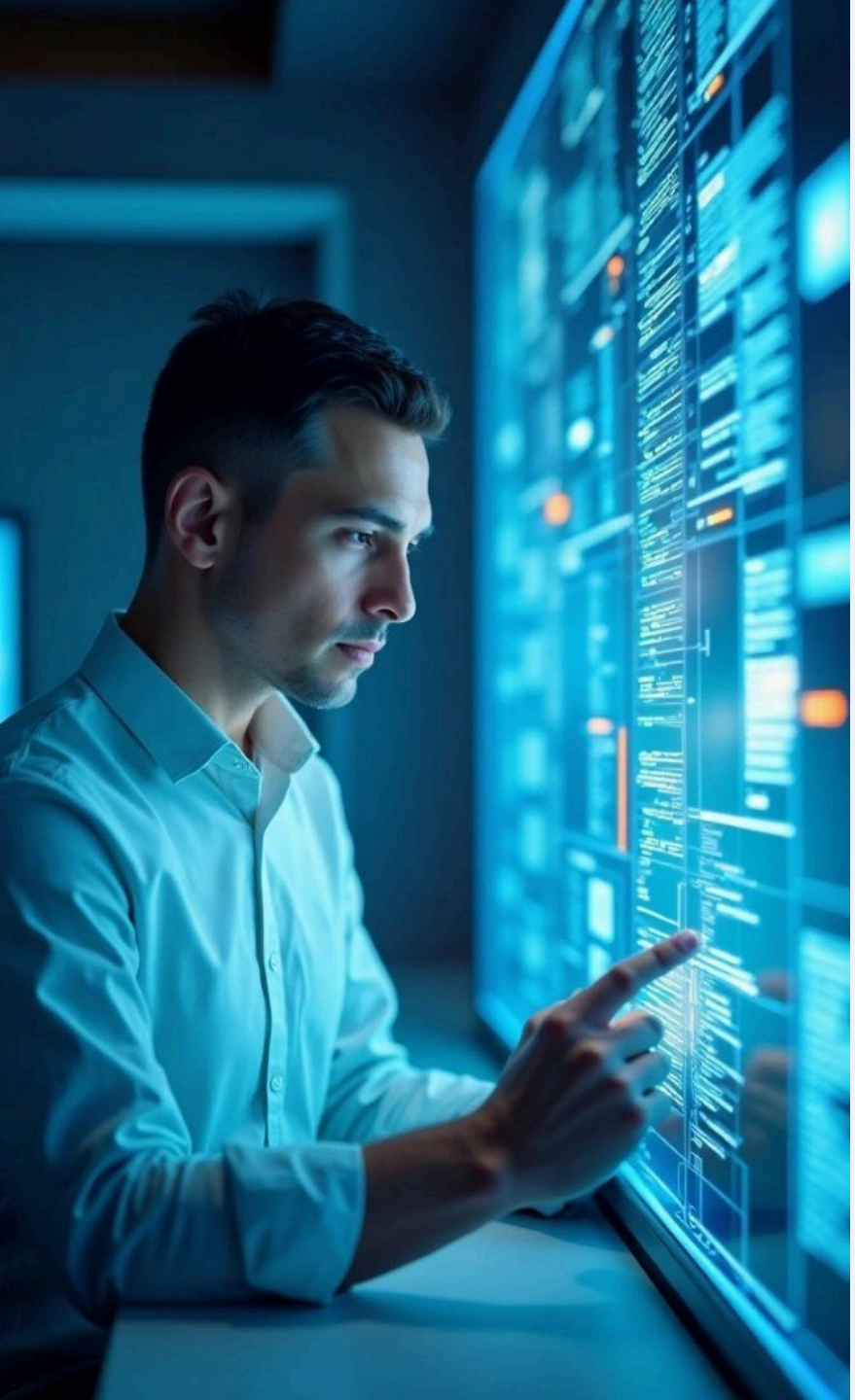
Selected Modules & Design

Presenter: **Salim Hamzaoui**

Selected Modules

Category	Module	Type	How it is implemented
Web	Use a framework as backend	**Major**	Django 4.2 + Django REST Framework; apps userapp, gameapp, tournaments
Web	Use a front-end framework or toolkit	**Minor**	Bootstrap 4.5 + custom CSS; vanilla-JS SPA router
Web	Use a database for the backend	**Minor**	PostgreSQL 13 via the Django ORM and migrations
User Management	Standard user management, authentication, users across tournaments	**Major**	Register / login, profiles, avatars, friends, stats, match history; tournaments of nicknames
User Management	Implementing a remote authentication	**Major**	42 Intra OAuth 2.0 (authorize → callback → server-side code exchange → JWT)
Gameplay & UX	Add another game with user history and matchmaking	**Major**	Tic-Tac-Toe: local or online — the queue pairs players by win rate, turn-based play, history for both
AI-Algo	Introduce an AI opponent	**Major**	PongAI: looks once per second, predicts the intercept incl. wall bounces, presses simulated arrow keys at player speed — no A*
AI-Algo	User and game stats dashboards	**Minor**	Profile cards, win-rate chart, match history, tournament scoreboard, JSON export
Cybersecurity	GDPR compliance: anonymization, local data management, account deletion	**Minor**	Anonymization (42-safe), data export, edit data, account deletion, inactive-account cleanup
Cybersecurity	Two-Factor Authentication (2FA) and JWT	**Major**	Email one-time code on login; SimpleJWT access / refresh tokens
Graphics	Use of advanced 3D techniques	**Major**	Three.js: perspective camera, lights, Phong materials, textured spinning ball
Accessibility	Expanding browser compatibility	**Minor**	Standard web APIs only (ES modules, fetch, WebGL); tested in Chrome, Edge and Firefox
Accessibility	Server-Side Rendering (SSR) integration	**Minor**	Django pre-renders the requested page, title/meta and the profile data; the SPA hydrates it

7 Major + 6 Minor (× 0.5) = 10 major-equivalents — 7 are required for 100 %. Not selected: support on all devices (responsive layout and touch remain features), remote players, live chat, microservices, multiple languages.



Technology Stack

Chosen for robustness, simplicity and the learning objectives of the curriculum — and because the subject fixes Django, Bootstrap, PostgreSQL and Three.js for the modules we selected.

Backend

Python 3.11, Django 4.2, Django REST Framework, SimpleJWT, python-decouple for configuration

Server

Gunicorn with TLS on port 443 (self-signed certificate), WhiteNoise for hashed static files

Database

PostgreSQL 13 in its own container, accessed through the Django ORM and migrations

Frontend

Vanilla JavaScript SPA, HTML5, CSS3, Bootstrap 4.5

3D graphics

Three.js r128 (WebGL) for the Pong scene; HTML5 canvas for procedural textures

DevOps

Docker Compose, Makefile targets, Git / GitHub with pull requests

System Architecture

The application is a monolith with a clear separation between frontend and backend, running in containers:

- The browser loads one server-rendered page and then runs the SPA.
- Gunicorn (3 workers) terminates HTTPS on port 443 and runs the Django app; WhiteNoise serves static files.
- Django exposes the REST API — accounts, games, Tic-Tac-Toe matchmaking, tournaments — and talks to PostgreSQL only through the ORM.
- External services: the 42 API for OAuth and Gmail SMTP for 2FA codes.
- Docker Compose defines the two services, the network and the database volume.

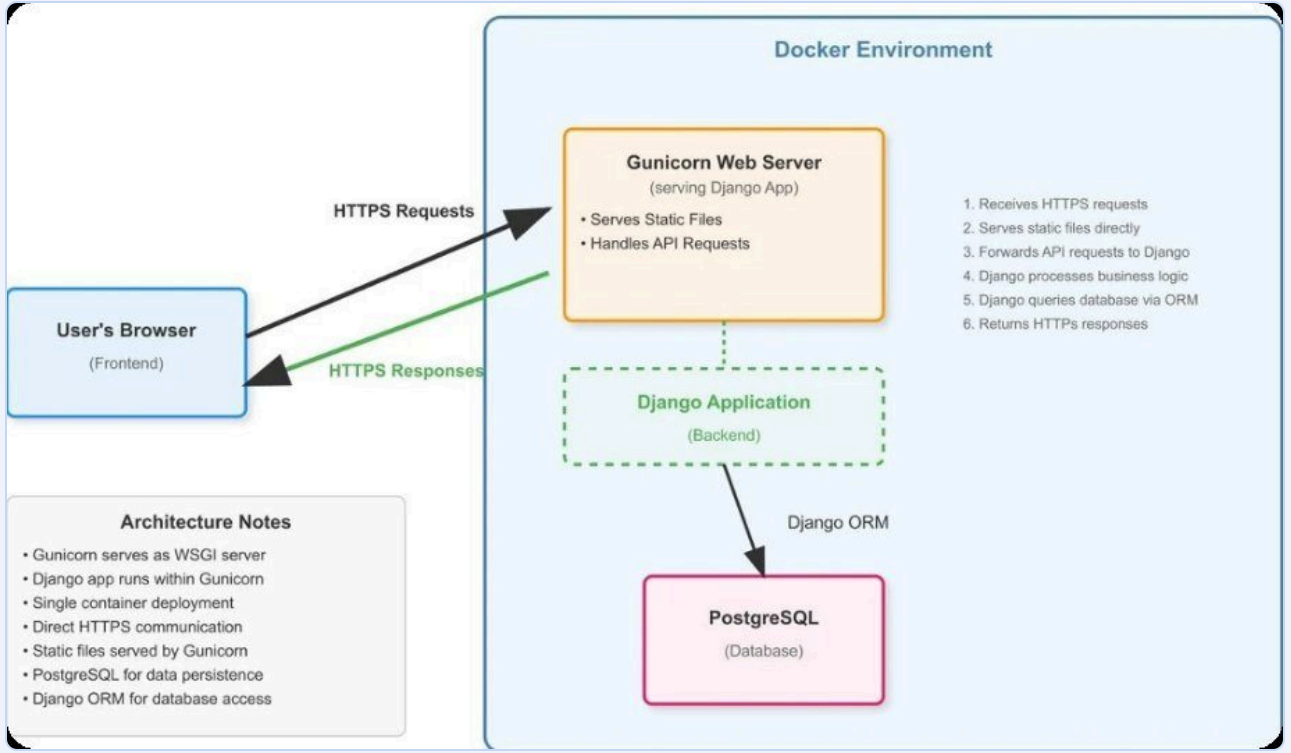
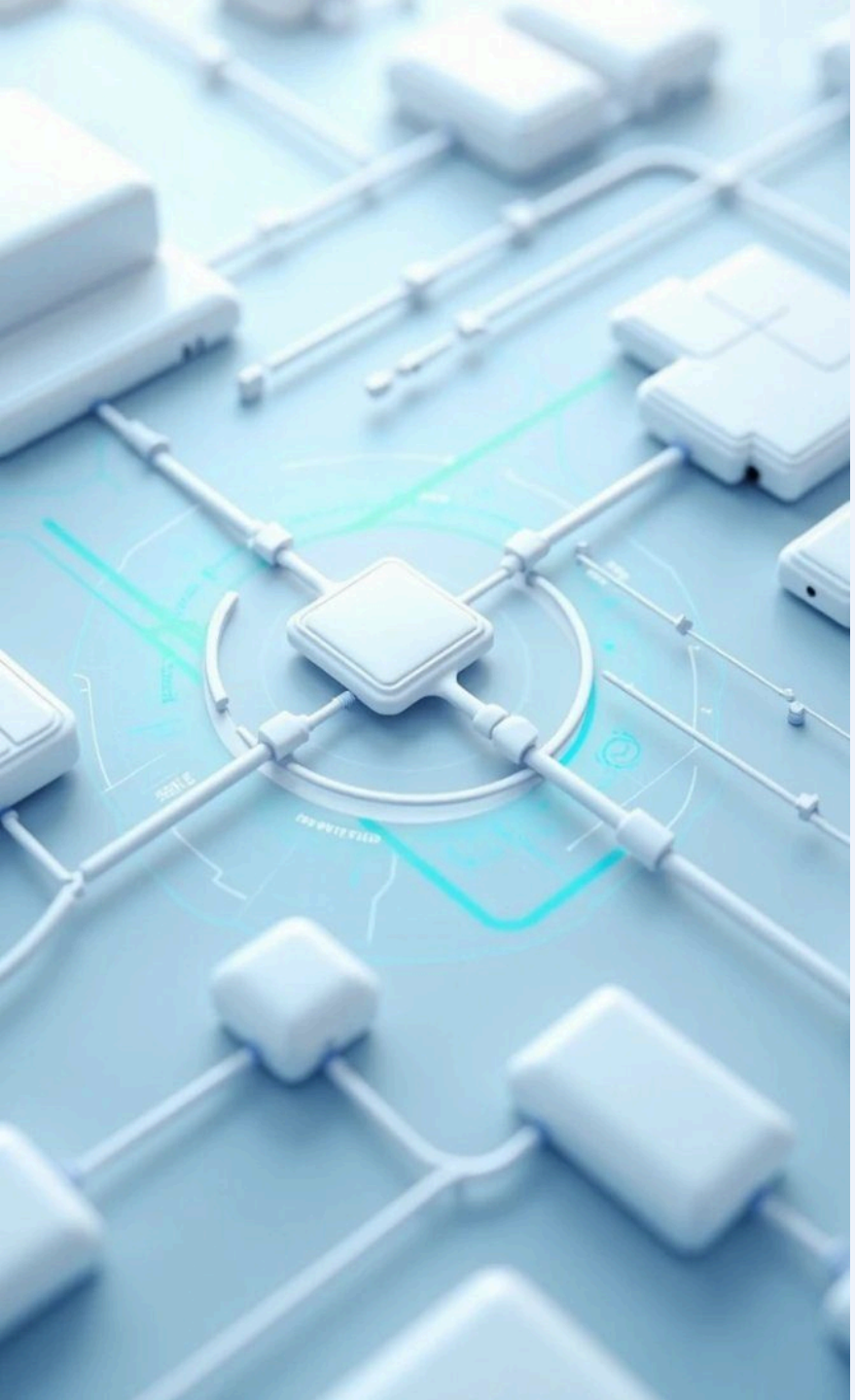


Figure 2: System architecture diagram



Database and API Design

The schema is defined with Django models, grouped into three apps; the frontend consumes a RESTful JSON API.

userapp

User (custom, email login, 2FA flag, avatar, friends M2M, last activity) and **MatchHistory** (game type, opponent, result, score, date).

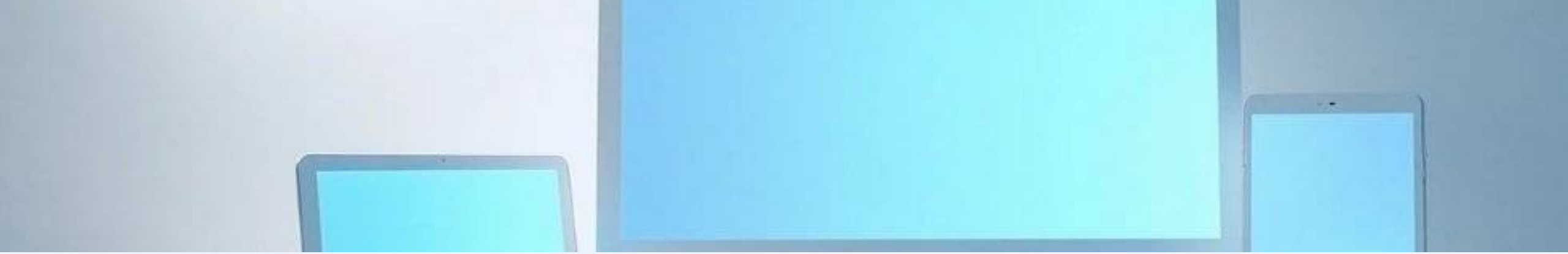
gameapp

TicTacToeQueue (waiting player + rating) and **TicTacToeMatch** (both players, 9-cell board, turn, status, winner).

tournaments

Tournament, **Player** (nickname per tournament) and **Match** (scores, winner, tiebreaker flag).

Endpoint	Purpose
POST /api/auth/register/ · login/ · logout/	Accounts and sessions (login starts 2FA)
POST /api/auth/verify-otp/ · token/refresh/	Check the emailed code, issue / refresh JWT
POST /api/auth/redirect_uri/ · get-token/	42 OAuth link and code exchange
GET/PUT /api/auth/profile/ · friends/… · users/	Profile, stats, avatar, display name, friends
/api/auth/save-match/ · match-history/	Record and list games
/api/auth/export-data/ · anonymize-account/ · delete-account/	GDPR tools
/api/game/ttt/queue/ · match/<id>/ · move/ · leave/	Tic-Tac-Toe matchmaking and online play
/tournaments/api/tournaments/…	Create, add players, view, start / finish matches



UI / UX Design

The interface is built around clarity, simplicity and responsiveness: one consistent colour scheme and layout, immediate feedback on actions, and an app-like SPA experience.

SPA experience

No full-page reloads: the client-side router swaps views and keeps browser history working.

Responsive

Flexbox / grid layouts and media queries adapt the pages from desktop monitors to phones.

Feedback

Buttons react on hover, loading states are shown during API calls, results and errors are displayed clearly.

Wireframes & flowcharts

Screens and the gameplay / tournament flows were sketched before implementation.

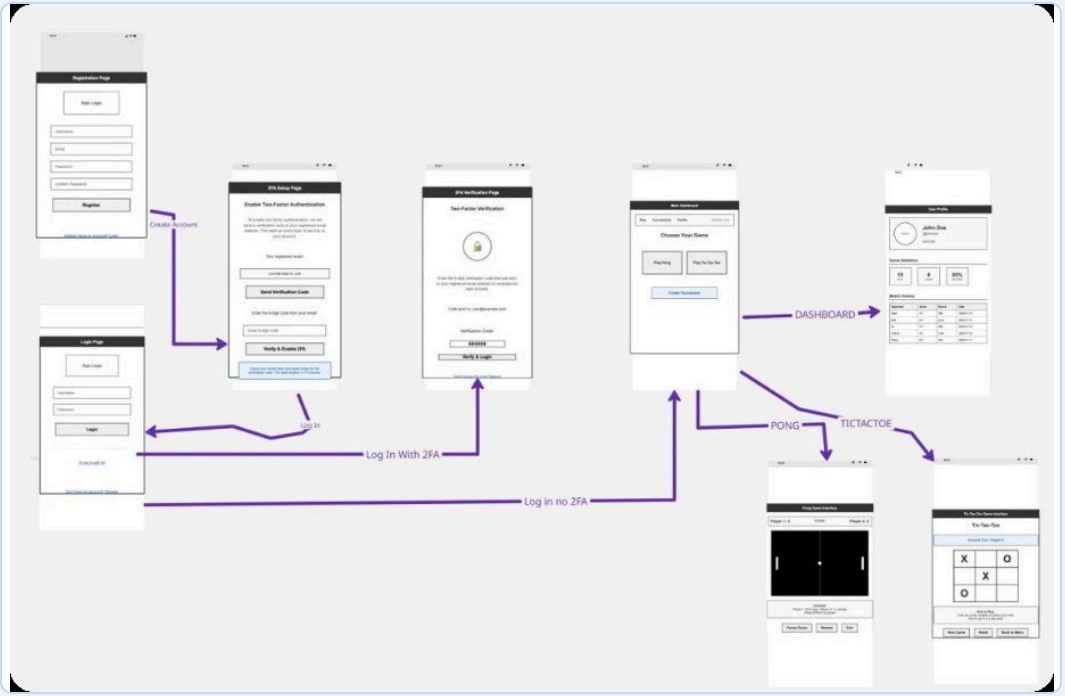


Figure 3: Wireframes of the key screens

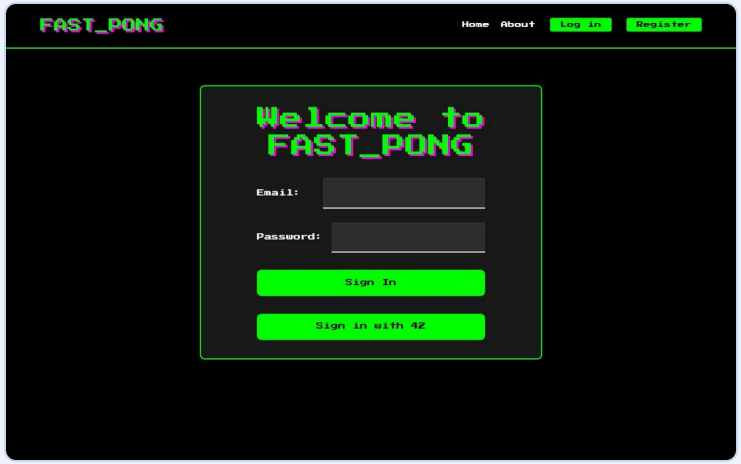
04

Authentication & Security

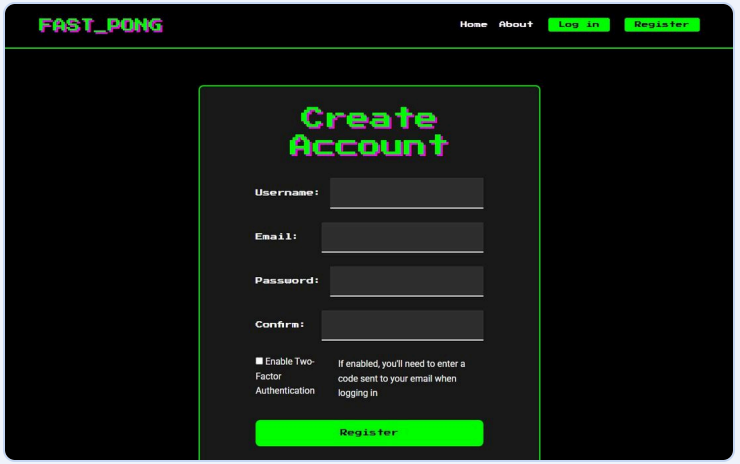
Presenter: **Nasser Alzaabi**

Features — Registration, Login and 2FA

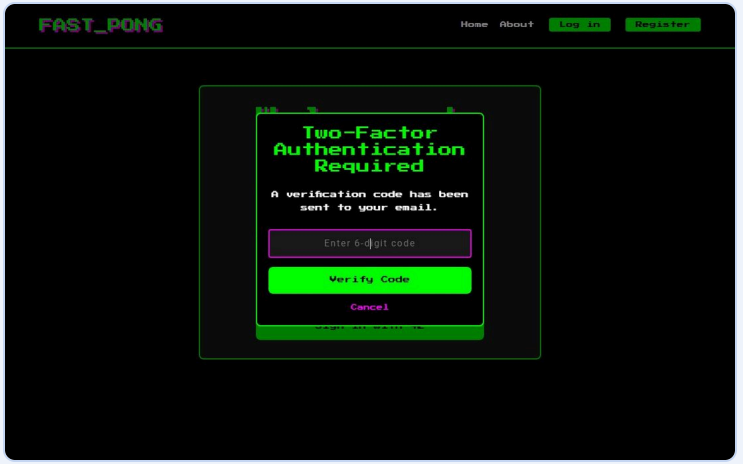
Email/password registration with a strong-password policy, “Sign in with 42”, and an emailed one-time code when 2FA is enabled (it can be switched on or off in Settings).



Login page: email / password or “Sign in with 42”



Registration with optional two-factor authentication



Second factor: 6-digit code sent by email, valid 10 minutes, single use

How a Login Works — 42 OAuth, 2FA and JWT

Three flows share one outcome: a logged-in user with a session and a JWT pair.

1 Password + 2FA

1. POST /login/ with email and password → Django `authenticate()` (PBKDF2 hash check).
2. If 2FA is on: a 6-digit code is stored in a **shared database cache** for 10 min and emailed from a background thread; the response is *requires_2fa*.
3. POST /verify-otp/ → code compared and deleted (single use) → session + JWT pair returned.

2 Remote authentication (42)

1. “Sign in with 42” → the backend builds the authorize URL (client id, redirect URI, *response_type=code*).
2. The student consents on the 42 Intra; 42 redirects to `/oauth/callback?code=…` in our SPA.
3. The SPA posts the code; the **server** exchanges it with the client secret (never in the browser), reads `/v2/me`, creates or links the user by email, logs in, returns JWTs.

3 JSON Web Tokens

1. SimpleJWT: access token 60 min, refresh token 7 days, signed HS256 with the Django secret; claims *user_id, exp, iat, jti*.
2. The SPA sends `Authorization: Bearer` on every API call (games, friends, matchmaking, GDPR).
3. `authFetch` refreshes the token one minute before expiry and retries once after a 401, so a session survives the 60-minute limit.

Both paths end in the same state: a Django session cookie (HttpOnly) plus a JWT pair — 42 users and password users are indistinguishable afterwards.

Cybersecurity Features

Security was a fundamental requirement, addressed in layers from the database to the browser.

Password storage

Django hashes and salts every password (PBKDF2); a validator enforces length, upper-case, digit and symbol.

2FA + JWT

Optional emailed one-time code as a second factor; SimpleJWT access (60 min) and refresh (7 days) tokens.

42 OAuth

Students log in through the 42 Intra; the server exchanges the code and never sees a password.

SQL injection

All database access goes through the ORM, which parameterises every query.

CSRF & XSS

Django’s CSRF token is required on every state-changing request; user data (names, nicknames) is inserted with textContent, never as HTML.

Transport & access control

HTTPS everywhere — Gunicorn terminates TLS on port 443; every game, tournament and profile API requires login; secrets live in .env.

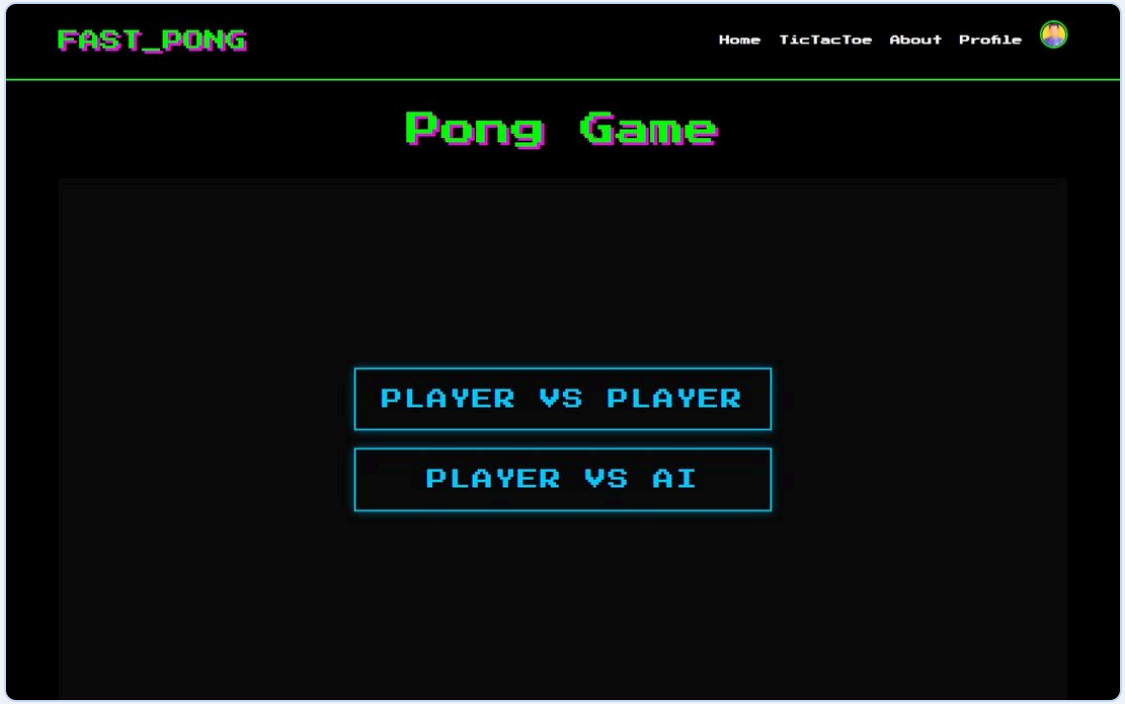
05

Games & Graphics

Presenter: **Salim Hamzaoui**

Features — 3D Pong and the AI Opponent

Three.js scene with a perspective camera, lit table and a spinning textured ball. In “Player vs AI” the PongAI looks at the ball once per second, predicts where it will cross the paddle line and presses simulated arrow keys — first to 3 points wins.



Mode selection: Player vs Player (one keyboard or touch) or Player vs AI

Game in progress against the AI

Inside the Graphics and the AI Opponent

3D scene (Three.js / WebGL)

PerspectiveCamera (75°) looking down at a glossy table with neon edge lines; ambient + spot lighting; Phong materials with specular highlights; emissive cyan paddles; a magenta ball whose stripes are painted on an HTML canvas and used as a texture; antialiased renderer; a DOM HUD over the canvas.

Physics

The bounce angle depends on where the ball hits the paddle (up to ±45°); the ball speeds up 5 % per hit; spin bends the trajectory and rotates the mesh; walls clamp and reflect; the canvas keeps a 4:3 ratio on resize; keyboard and touch input.

AI — look once per second

Every 1000 ms the AI takes one snapshot of the ball position and velocity (the subject's "refresh once per second" rule). Between snapshots it acts only on its last decision.

AI — predict, with errors

It extrapolates where the ball will cross its paddle line, folding the path at the walls so bounces are anticipated; then adds a small prediction error and, 10 % of the time, a big "mistake".

AI — simulated keyboard

The decision becomes a held ArrowUp / ArrowDown key fed into the same InputHandler as a human, so the AI paddle moves at exactly human speed. Every 5 s the score adjusts its judgement: it eases off when 2 points ahead, tries harder when behind. No A* — Pong has no graph to search.

Features — Tic-Tac-Toe with Online Matchmaking

The “Add another game” module: a new game distinct from Pong, with user history tracking and a matchmaking system.

Mode selector: Local (2 players, one keyboard) or Online — find an opponent

The 3 × 3 board during a game

Local mode

Two players on one keyboard/mouse; the result is saved to the match history of the logged-in user.

Matchmaking

“Find an opponent” puts the user in a queue with a rating = their Tic-Tac-Toe win rate. The server pairs the two closest ratings (fair matches); entries older than 60 s are dropped; the browser polls every 2 s.

Online play, server-side rules

The board lives in the database. Every move is validated on the server — your turn, free cell — win lines and draws are detected there, and both players poll the state every second. Leaving a match is a forfeit.

User history

When a match ends the server writes one MatchHistory row per player (WIN / LOSS / DRAW, opponent’s name), so it appears on both profiles, in the win-rate chart and in the JSON export.

06

Tournaments

Presenter: **Nour Murat**

Features — Tournaments

A logged-in user creates a tournament of 3–8 nicknames; every pair plays one Pong match, scores update the table, and tied leaders get automatic tiebreaker matches until one winner remains.

Create a tournament and register the nicknames (unique inside the tournament)

Round-robin schedule with “Start Match” per pairing, “Next match” announcement, live scores and the winner

Tournament Flow

From creation to the final winner, the tournament system runs six steps:

1 · Create

A logged-in user names the tournament and chooses 3–8 players.

2 · Register

Each player types a nickname; duplicates and empty names are rejected.

3 · Schedule

The server generates the round-robin: every pair meets exactly once.

4 · Play

“Start Match” opens 3D Pong with both nicknames on screen (two players, one keyboard).

5 · Score

The result is posted to the tournament API; the table shows points per player and announces the next match.

6 · Winner

If leaders are tied, extra tiebreaker matches are created until one player wins.

Users across tournaments

Nicknames belong to one tournament, so a person can join many tournaments with different aliases; the account that created the tournament must be logged in.

Kept safe

All tournament endpoints require login and the CSRF token; nicknames are shown as text (no HTML injection); scores must be non-negative integers and a match cannot end in a tie.

Stays after refresh

The current tournament id is remembered in the browser, so reloading the page returns to the same tournament.

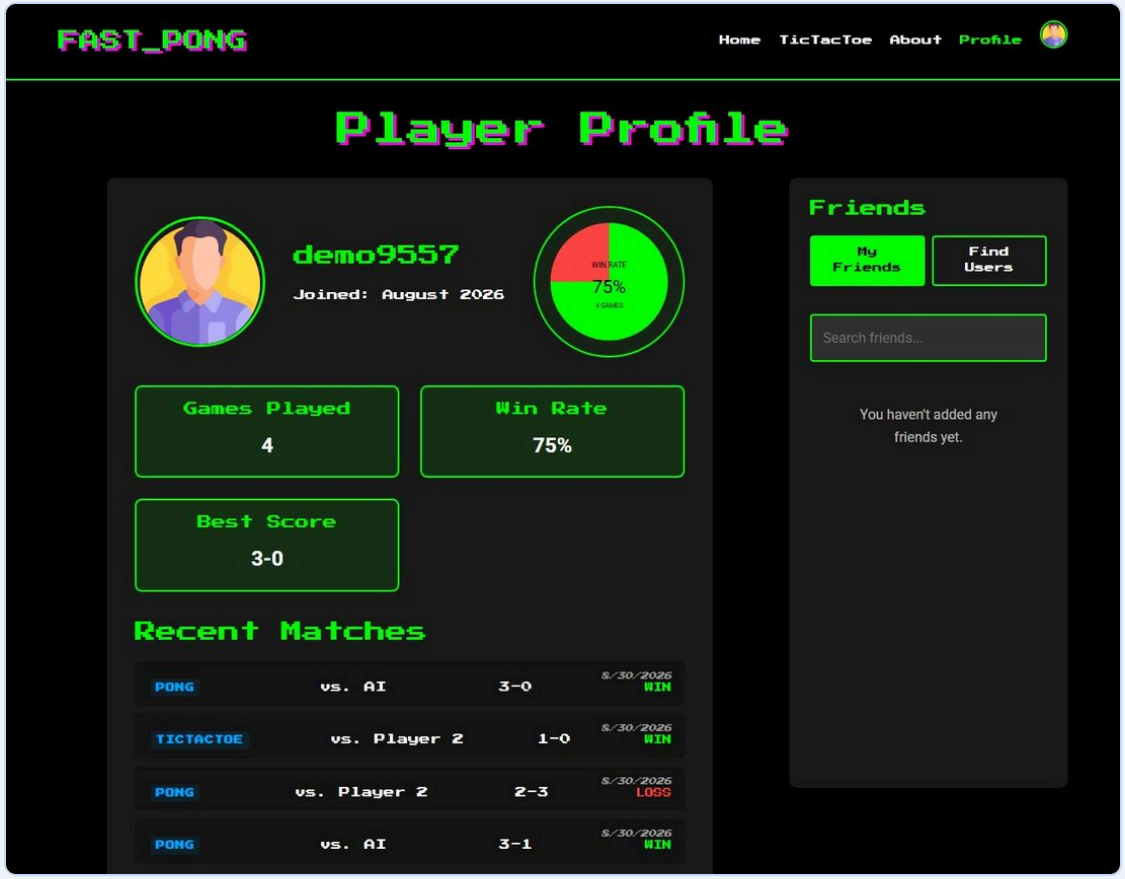
07

Profiles, Statistics & Friends

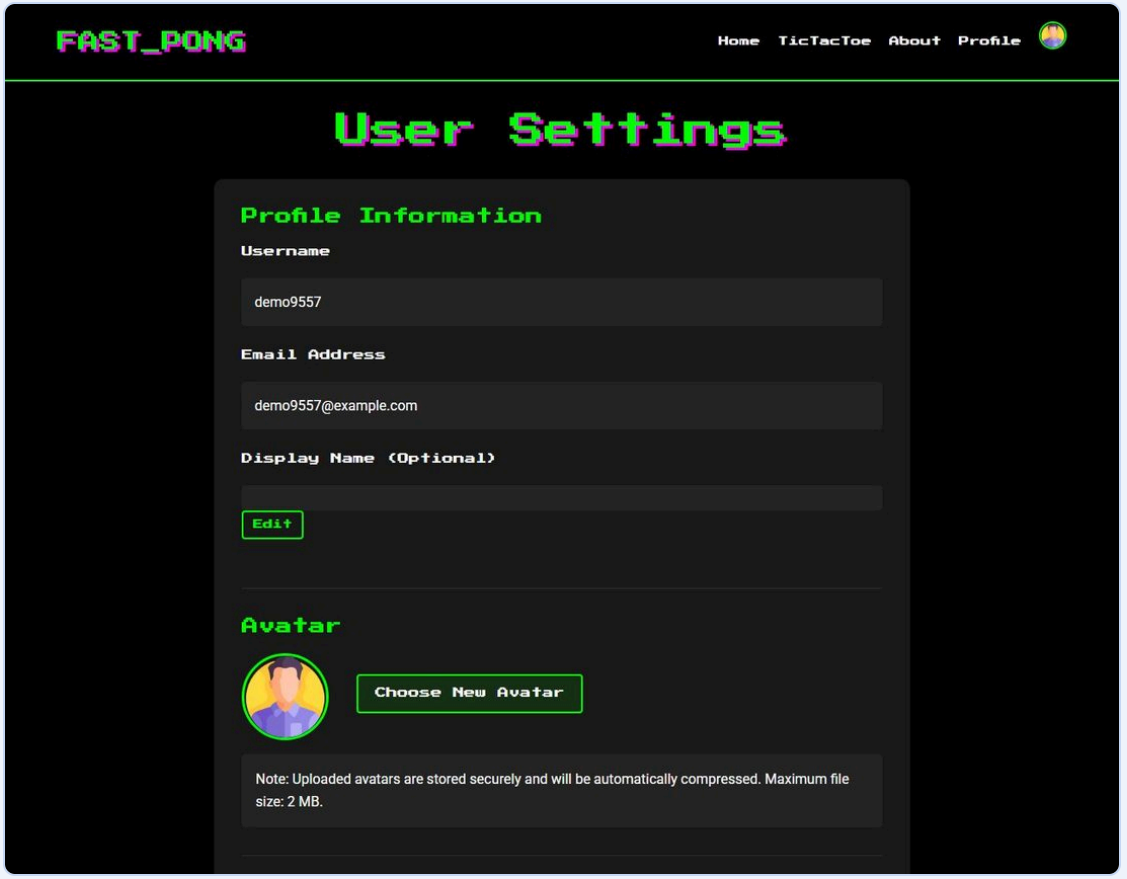
Presenter: **Alisher Abdullaev**

Features — Player Profile and Stats Dashboard

Every finished game — Pong, Pong vs AI, Tic-Tac-Toe local or online — is recorded; the profile turns the history into a dashboard and the settings page holds the account and GDPR tools.



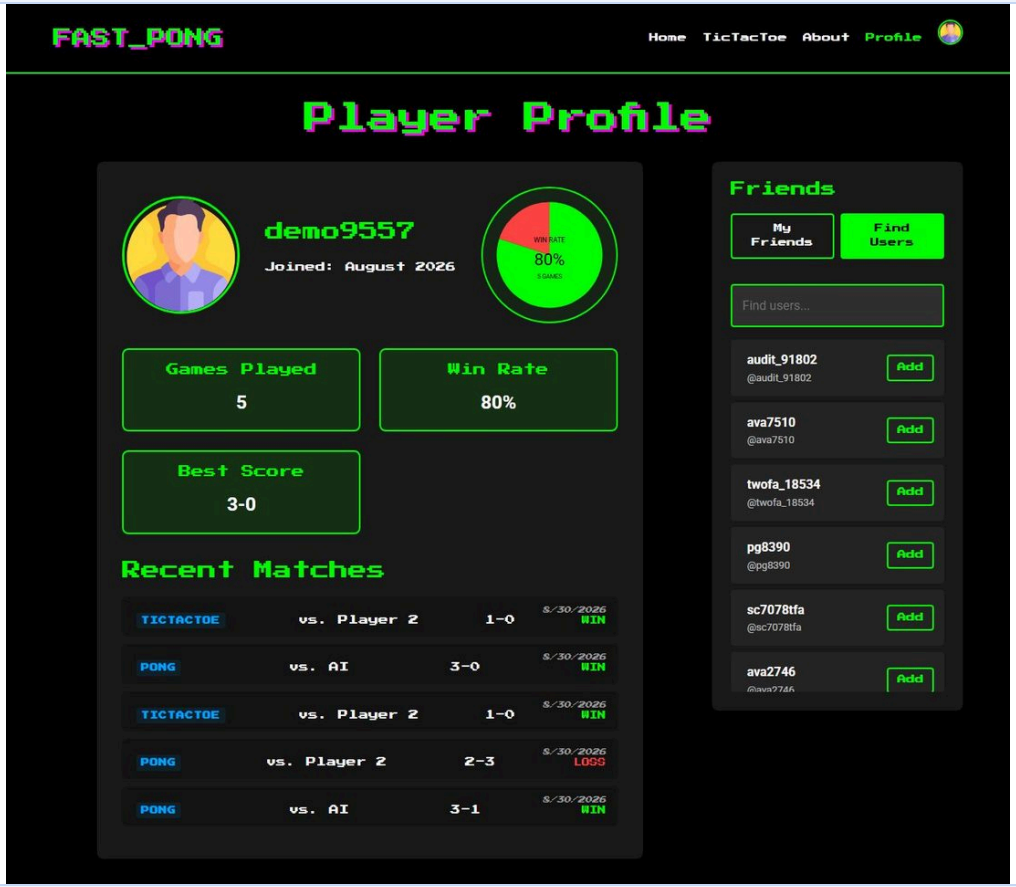
Games played, win-rate chart, best score, recent matches with game type and date, friends panel



Display name, e-mail, avatar, 2FA switch, Download my data, Anonymize, Delete account

Features — Friends, Match History and User Data

The user-management and stats-dashboard modules, seen from the user's side.



"Find Users" lists every active player with an Add / Remove friend button

User information

Custom Django user: e-mail is the login, unique username, optional display name, avatar (default picture if none, max 2 MB, image type checked), 2FA flag, 42 link.

Friends

Add or remove friends from the Find Users list; the friends panel on the profile shows their names and avatars.

Match history

One row per game: type (PONG / TICTACTOE), opponent, score, result, date. Last five on the profile, full list in the JSON export.

Statistics

Games played, win rate (hand-drawn SVG pie chart), best score = the win with the largest margin. Tournament games are kept separate because their players are nicknames, not accounts.

08

GDPR & Accessibility

Presenter: **Salim Hamzaoui**

GDPR Compliance

Users own their data: they can see it, take it with them, anonymize it or erase the account — and inactive accounts are cleaned up automatically.

Anonymization

“Anonymize My Account” replaces username, e-mail, avatar and 42 link with anonymous values, clears friends and disables login; the non-personal game statistics are kept. Works for 42 accounts too.

Local data management

“Download my data” exports profile, statistics and full match history as JSON; display name, e-mail and avatar can be edited in Settings.

Account deletion

Hard-deletes the account and everything attached to it (match history, friend links, tokens) in one click, after confirmation.

Retention

A management command warns after 5 months of inactivity and deletes after 6 (last activity tracked by middleware).

The privacy policy on the About page describes the data collected, its use, retention and the user’s rights.

Accessibility — Browser Compatibility and Server-Side Rendering

Expanding browser compatibility

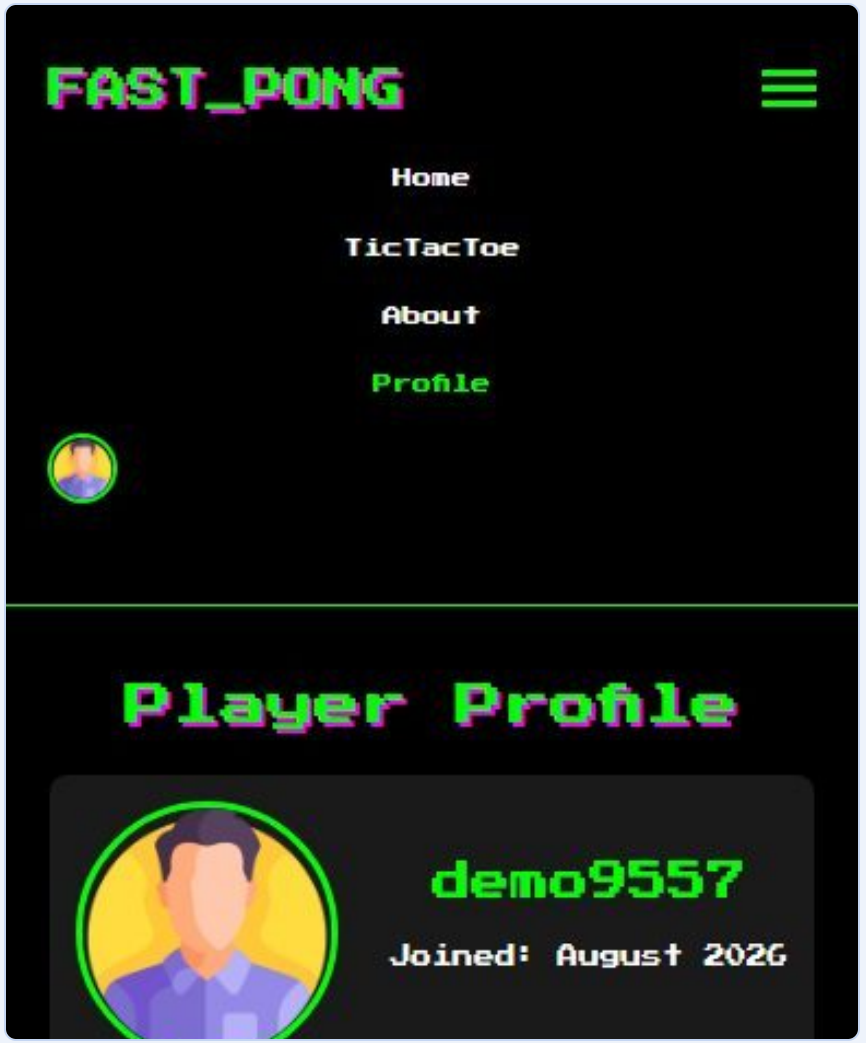
Primary browser: Chrome (and Edge). Additional browser: Firefox. The app uses only standard web APIs — ES modules, fetch, localStorage, History API, CSS Grid / Flexbox, WebGL 1 — with no vendor prefixes or polyfills. Browser-specific fixes: ISO-8601 dates so Firefox parses them, pointer events instead of touch/mouse events, font fallbacks. Tested manually in both browsers and with an automated headless-Chrome walkthrough of every page.

Server-Side Rendering

Every URL is answered by Django's `index` view, which renders the *requested* page as complete HTML: the right section is already active, the navigation reflects the login state, the page title and meta description are set, and for a logged-in user the profile — username, statistics, recent matches — is rendered into the HTML before any JavaScript runs. The SPA then hydrates and takes over routing. View-source shows real content, not an empty `<div>`, which helps first paint and SEO.

Responsive layout (feature)

Media queries at 1100 / 920 / 768 / 480 px, hamburger navigation, stacked cards, a fluid game canvas and touch controls for Pong — kept as a feature although "support on all devices" is not a claimed module.



Profile on a 390 px phone

09

Testing & Evolution

Presenter: **Nasser Alzaabi**

Testing Strategy

Several levels of testing, each with its own focus, keep the application stable and secure.

Unit tests

Django test suite (`make test`) — 62 tests across userapp, gameapp and tournaments: login and the whole 2FA flow, GDPR export / anonymize / delete, the inactive-account command, tournament tiebreakers, Tic-Tac-Toe matchmaking and moves, server-side rendering.

Integration tests

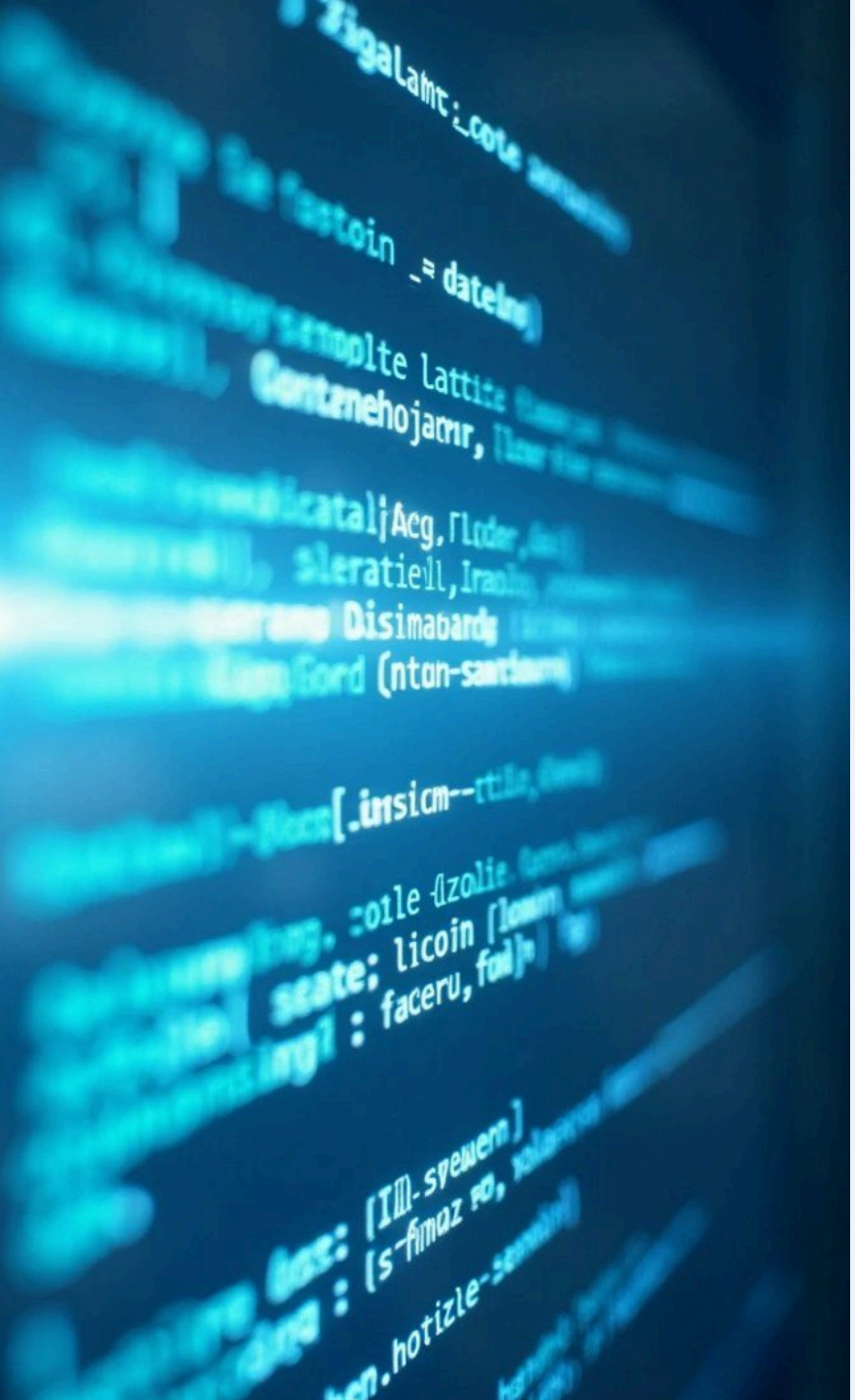
Scripted end-to-end API flow: register → login → profile → matches → friends → export → tournament → delete.

Browser walkthrough

Headless Chrome drives every page on desktop and phone, plays both games and checks for JavaScript errors.

Manual / acceptance

Team members continuously tested each other’s features from an end-user perspective, in Chrome and Firefox.



Pre-Evaluation Audit (August 2026)

Before the evaluation the codebase was audited end-to-end. Two bugs reported by users were traced to their root causes and fixed with regression tests:

“The 2FA code is sometimes rejected”

The one-time code was kept in Django’s default in-memory cache, which is private to each process — and Gunicorn runs three workers. The verification request often reached a worker that had never seen the code. Fix: a database-backed cache shared by all workers.

“The 2FA e-mail is very slow”

The e-mail was sent synchronously inside the login request with no timeout, so the response waited for the whole SMTP round-trip (and failed with 500 on any error). Fix: send in a background thread with a 10 s timeout — login now answers in ~80 ms.

Also fixed

`make test` configuration, stale static files, the token-refresh URL in the SPA, a settings-page bug that saved a placeholder display name, the Pong ball gliding along a wall (bounce now clamps the ball and keeps a minimum angle), and the previous account's avatar remaining visible after switching users.

Result

A second sweep found and fixed 30 more bugs (silent JWT expiry after 60 min, secrets in logs, duplicate-registration errors, tournament persistence and repeated tiebreakers, Save Settings, 2FA toggle, Pong resize / touch / pause leaks, input validation). A third pass checked every module against the subject: stored XSS via tournament nicknames fixed, AI presses simulated keys at player speed and anticipates bounces, DB credentials moved to .env, tournament API requires login, next-match announcement, GDPR anonymization, online Tic-Tac-Toe matchmaking, real SSR. 54/62 tests pass, 0 JavaScript errors.

Challenges and Lessons Learned

Building a complete application from scratch provided technical and organisational hurdles — and lasting lessons.

Tournament logic

Generating fair schedules and resolving ties correctly required careful data modelling and state management.

State in vanilla JS

Without a framework, login state, routing and views had to be managed by hand with disciplined code structure.

Asynchronous flows

OAuth redirects, 2FA, matchmaking polling and API calls demanded a solid grasp of Promises and loading states.

Docker networking

Getting the web and database containers, volumes and environment variables to cooperate took iteration.

Multi-process bugs

The 2FA cache bug only appears with several Gunicorn workers — a lesson in testing on the real deployment.

Lessons

A well-defined API, a mature framework, a disciplined Git workflow and security from day one all paid off.

Limitations and Future Enhancements

The current version meets every selected module; the following points are known limitations and the improvements we would make next.

Known limitations

Pong is local multiplayer only (online play exists for Tic-Tac-Toe); matchmaking and online moves use 1–2 s polling rather than WebSockets; JWT stored in localStorage; the 42 OAuth flow has no *state* parameter; no rate limit on the 2FA code; third-party assets load from CDNs; the 2FA mailbox needs a valid Gmail app password; Pong scores are reported by the client.

Next steps

1. Real-time online Pong and live chat with WebSockets (Django Channels). 2. OAuth *state* parameter and rate limiting on login / 2FA. 3. Server-authoritative Pong scores. 4. Selectable AI difficulty. 5. Leaderboards, achievements and 2FA recovery codes.

10

Team & Conclusion

Presenter: **Alisher Abdullaev**

Contribution of Each Member

Each member owned a group of modules end-to-end — backend, frontend and tests — and reviewed the others' pull requests.

Salim Hamzaoui

Graphics — 3D Pong with Three.js and its physics; **Gameplay** — the second game, Tic-Tac-Toe, with online matchmaking and history; **Cybersecurity & GDPR** — anonymization, data export, account deletion, retention cleanup, privacy policy.

Nasser Alzaabi

Authentication — registration, login, sessions; **Remote authentication** — the 42 OAuth 2.0 flow; **2FA and JWT** — emailed one-time codes, SimpleJWT access / refresh tokens and their automatic renewal in the SPA.

Alisher Abdullaev

User history and statistics — match history, stats dashboard, win-rate chart; **user information** — username, avatar, display name, e-mail, friends; **Database module** — PostgreSQL, models and migrations.

Nour Murat

Mandatory part — the tournament system with round-robin scheduling and tiebreakers; **Front end** — the SPA and the Bootstrap toolkit module; **Backend framework module** — the Django project structure and REST API.

Conclusion

Ft_transcendence delivered a feature-rich, secure and engaging web gaming platform that satisfies all the selected modules — 7 Major and 6 Minor, 10 major-equivalents against the 7 required.

Using Django, a vanilla-JavaScript SPA with Bootstrap, Three.js and PostgreSQL inside Docker, the team gained hands-on experience in full-stack development, online game logic, deployment and application security. The Agile, feature-branch workflow kept four developers productive and the codebase reviewable.

The modular structure is a solid base for the next evolution — real-time online Pong, chat and richer AI — while the pre-evaluation audit leaves the project with a green test suite and documented, root-caused fixes.

Thank You